

polimorfism

Șapte exerciții — aceeași buclă, obiecte care răspund diferit · TypeScript · Andreea

Unde lucrezi: `vanilla-ts/src/exercitii/`, peste ce ai făcut la fișa de moștenire. Două exerciții cer un folder nou în `vanilla-ts/src/fise/`; acolo copiezi clasele de care ai nevoie.

Nou, ai voie: tipul de bază ca tip al variabilei, al array-ului sau al parametrului; `override`; `super`.

Interzis pe toată fișa: `instanceof`, `typeof`, `if / switch` după numele sau tipul obiectului, `as`, `any`. **Încă NU folosim:** `abstract` nou, `interface`.

Cum se lucrează: fiecare exercițiu îți cere să **construiești** ceva. Rândul **Gata când** spune cum verifici; se verifică prin rulare și prin `npx tsc --noEmit`. Unde scrie `git diff`, chiar rulezi comanda și te uiți ce linii apar.

Partea 1 — intră în listă fără să atingi bucla

1. O verificare nouă

nivel 1

Fișierul `05-verificari`. Adaugă **IncepeCu**, care primește un prefix și trece dacă valoarea începe cu el. Pune-o în array, la final.

Rulare: prefixul `ESID-`, aceleași trei valori.

Gata când: fiecare linie are acum patru verificări, ultima fiind `IncepeCu`: OK pentru `ESID-100` și FAIL pentru celelalte; iar `git diff` pe fișier arată doar clasa nouă și linia din array. Buclea nu apare în diff.

De reținut: bucla n-a aflat că există un tip nou. Asta e tot polimorfismul: codul care folosește nu știe cine îl folosește.

2. Markdown la export

nivel 1

Fișierul `06-exportatoare`. Adaugă **ExportatorMarkdown**, extensia `md`, fiecare rând de forma `| ESID-100 | Active |`. Pune-l în array.

Gata când: după `raport.txt` apare `raport.md` cu cele două rânduri, iar bucla de export nu apare în `git diff`.

De reținut: ai adăugat un format de fișier fără să atingi codul care exportă.

3. Funcția care primește baza

nivel 1

Fișierul `05-verificari`. Scrie o funcție `careNuTrec(verificari: Verificare[], valoare: string): string[]` care întoarce numele verificărilor picate pentru valoarea dată.

Rulare: pe `" "` și pe `ESID-100`, cu array-ul de la exercițiul 1.

Gata când: pentru `" "` ies toate patru numele, pentru `ESID-100` iese doar `EDinLista`; funcția are o singură buclă sau un singur `filter` și niciun nume de verificare scris în ea.

De reținut: parametrul de tip `Verificare` acceptă orice subclasă, și pe cele scrise azi, și pe cele de mâine.

Partea 2 — polimorfismul face muncă

4. Raportul din spatele variabilei

mai greu

Fișierul `03-raport`. Pune într-un array de tip `Raport[]` un `Raport`, un `RaportSemnat` și un `RaportCuData`, cu aceleași două linii fiecare. O singură buclă afișează `text()` pentru fiecare, cu o linie `===` între ele.

Apoi scrie `const r: Raport = new RaportSemnat("T", "andreea");`. Înainte să rulezi, scrie în răspuns ce crezi că afișează `r.text()`: cu sau fără linia `-- andreea`? Abia apoi rulează.

Gata când: predicția și rezultatul sunt scrise amândouă în răspuns și coincid, iar tu explici într-un rând de ce tipul scris în stânga nu a contat.

De reținut: metoda se alege după obiectul construit cu `new`, nu după tipul declarat al variabilei. Tipul declarat spune doar ce ai voie să chemi.

5. Validarea unui set de date

mai greu

Folder nou: `src/fise/validare-set/main.ts`. Copiază acolo `SetDeDate` (de la 01) și verificările (de la 05, cu `IncepeCu`).

Scrie funcția `valideaza(set: SetDeDate, reguli: Map<string, Verificare[]>): string[]`: pentru fiecare coloană din `reguli`, în ordinea din `Map`, pentru fiecare rând, fiecare verificare; la fiecare picare produce un mesaj `rand 2, ESID: AreLungimea FAIL` (rândurile numărate de la 0).

Rulare: coloanele `ESID`, `STATUS`; rândurile `ESID-100 / Active`, `ESID-777 / pending`, `ESID-78 / Active`; regulile: `ESID → AreLungimea(8)`, `IncepeCu("ESID-")`; `STATUS → NuEGoala`, `EDinLista([...])` cu cele patru stări.

```
rand 2, ESID: AreLungimea FAIL
rand 1, STATUS: EDinLista FAIL
```

Gata când: ies exact cele două linii de mai sus; în funcție nu apare niciun nume de verificare; iar când adaugi `AreLungimea(6)` la `STATUS`, apare o a treia linie fără să fi atins funcția.

De reținut: asta e testul de date pe care îl faci la serviciu, în mic: regulile sunt obiecte, iar funcția care le aplică nu știe ce reguli sunt.

Partea 3 — provocări

6. Negația

provocare

Fișierul `05-verificari`. Adaugă **Negata**, care primește la construire o altă verificare și trece exact când aceea nu trece. Numele ei e `Nu` + numele celei dinăuntru. Pune în array `new Negata(new EDinLista([...]))`.

Gata când: coloana `NuEDinLista` are OK/FAIL inversate față de `EDinLista` pe toate cele trei valori; `Negata` nu conține numele niciunei verificări; iar `new Negata(new Negata(x))` dă același rezultat ca `x`.

De reținut: o verificare care conține o verificare. Polimorfismul merge și în interiorul obiectului, nu doar în bucla din `main`.

7. Varianta fără polimorfism

provocare

Folder nou: `src/fise/fara-polimorfism/main.ts`. Rescrie exercițiul 5 fără subclase: o singură clasă **Regula** cu un câmp `tip: string ( "nuEGoala", "lungime", "dinLista" )` și parametrii necesari, iar funcția `valideaza` alege cu `if / else if` după `tip`. Aici `if` -ul după `tip` e permis: asta e scopul.

Apoi adaugă regula `incepeCu` **în ambele variante** (cea de la 5 și cea de aici) și numără liniile atinse în fiecare, cu `git diff --stat`.

Scrie trei rânduri: unde s-a dus `if` -ul în varianta polimorfică?

Gata când: ambele variante dau aceeași ieșire, iar cele două numere de linii atinse sunt scrise în răspuns.

De reținut: polimorfismul nu elimină decizia. O mută din funcția care folosește în obiectul care știe, lângă datele lui.